AJ_24038155_dialect_v1.2_to_v1.3 update

Change specification from AJ_24038155 Dialect v1.2 to v1.3

Status

This document records the v1.2 to v1.3 update to the frozen AJ_24038155 Dialect v1.2 specification. All v1.0, v1.1 and v1.2 lexical, syntactic and semantic rules remain normative unless explicitly changed below.

Version v1.3 is introduced under the v1.2 versioning rule because two new explicit language capabilities are required by the study corpus: runtime primitive type conversion and whole-program termination. The additions are deliberately narrow and do not introduce implicit coercion, runtime type introspection, substring slicing, callbacks, first-class function values or anonymous functions.

1. Explicit runtime casts

1.1 Canonical syntax

Runtime conversion uses `CALL` followed by a primitive cast target:

```
CALL INTEGER(expression)
CALL FLOAT(expression)
CALL STRING(expression)
```

The preceding `CALL` is semantically significant and distinguishes a runtime cast from the existing typed-literal constructors:

```
INTEGER(23)               COMMENT typed integer literal
CALL INTEGER(value)       COMMENT runtime conversion

FLOAT(3.5)                COMMENT typed float literal
CALL FLOAT(value)         COMMENT runtime conversion

STRING("23")              COMMENT string literal
CALL STRING(value)        COMMENT runtime conversion
```

1.2 Grammar extension

```
cast_expression ::=
    "CALL" , cast_type , "(" , expression , ")" ;

cast_type ::=
    "INTEGER"
  | "FLOAT"
  | "STRING" ;
```

`cast_expression` is added to `primary_expression`. Each cast accepts exactly one argument.

1.3 Runtime semantics

`CALL INTEGER(expression)` converts the evaluated value to an integer using deterministic Python-compatible runtime conversion semantics. Invalid conversions raise a runtime error and may be handled by an enclosing AJ `TRY ... CATCH`.

`CALL FLOAT(expression)` converts the evaluated value to a floating-point value using deterministic Python-compatible runtime conversion semantics. Invalid conversions raise a runtime error.

`CALL STRING(expression)` converts the evaluated value to its textual representation using deterministic Python-compatible runtime conversion semantics.

The compiler must not insert any of these conversions implicitly. A cast occurs only where the AJ source explicitly contains the corresponding `CALL` expression.

1.4 Python translation

```
CALL INTEGER(value) -> int(value)
CALL FLOAT(value)   -> float(value)
```

```
CALL STRING(value)  -> str(value)
```

Example:

```
TRY
    givenNum <- CALL INTEGER(givenNum)
CATCH
    OUTPUT STRING("Invalid integer")
ENDTRY
```

The conversion remains inside the generated Python `try`/`except` region so conversion failure remains observable and catchable.

1.5 AST and flow representation

A cast is represented explicitly as a `CastNode` containing the target type and source expression. This preserves conversion intent for AST, flow-diagram and scoring/audit stages rather than disguising a cast as an ordinary user-defined function call.

2. Whole-program termination with QUIT

2.1 Canonical syntax

```
QUIT
```

`QUIT` is a statement and may appear at top level or inside an executable block.

2.2 Semantics

`QUIT` terminates execution of the current program immediately. It is distinct from `RETURN`, `BREAK` and `CONTINUE`. A top-level Python `quit()` is therefore represented canonically by `QUIT`, not by a top-level `RETURN`.

2.3 Grammar extension

```
quit_statement ::= "QUIT" , newline ;
```

`quit_statement` is added to `statement`.

2.4 Python translation

```
QUIT -> quit()
```

2.5 Control-flow semantics

A `QUIT` node is terminal. The flow diagram must not connect it to a following executable statement or route it back to a loop decision. The AST uses a dedicated `QuitNode` so explicit termination remains distinguishable from `RETURN`, `BREAK`, `CONTINUE` and historical `STOP` compatibility syntax.

3. Reserved identifier interaction

`INTEGER`, `FLOAT`, `STRING` and `QUIT` are reserved AJ keywords. The established source-proven restricted-identifier escape rule remains applicable when source syntax unambiguously proves that one of these spellings is being used as an identifier binding.

For example:

```
quit <- BOOLEAN(FALSE)
```

may be normalised to the established protected internal identifier form, while a standalone:

```
QUIT
```

retains its whole-program termination meaning. The normaliser must not rename constructor, cast or statement uses.

4. Static semantic additions

1. A cast occurs only when explicitly present in AJ source; implicit type coercion must not be invented.
2. `CALL INTEGER`, `CALL FLOAT` and `CALL STRING` accept exactly one argument.
3. Invalid INTEGER/FLOAT conversions are runtime errors and remain catchable by `TRY ... CATCH`.
4. Typed literal constructors retain their existing meaning and are not reinterpreted as runtime casts.
5. `QUIT` is valid at top level and inside executable blocks.
6. `QUIT` terminates the complete program and is not equivalent to `RETURN`, `BREAK` or `CONTINUE`.
7. `QUIT` must be represented as a terminal control-flow operation.

5. Backwards compatibility

All canonical AJ v1.2 programs remain valid under v1.3.

The historical `STOP` compatibility statement remains accepted for existing corpus material, but new canonical v1.3 pseudocode should use `QUIT` where the intended semantics are whole-program termination corresponding to Python `quit()`.

Typed literal constructors remain unchanged. In particular, `INTEGER(variable)` is not a runtime cast; runtime conversion requires the explicit `CALL INTEGER(variable)` form.

Callbacks, first-class function values, anonymous functions and nonlocal-style rebinding remain outside the dialect.

6. Normalisation rules

Conservative v1.3 canonicalisation must preserve explicit casts and `QUIT` statements. It must not insert a cast or program-termination operation that the source did not contain.

Where compatibility processing applies the source-proven restricted-identifier escape rule, the transformation must be logged and must preserve constructor, cast and statement meanings.

All earlier semantic-preservation restrictions remain normative.

7. Version identification

This specification delta is frozen as:

Cambridge-Style Pseudocode - AJ_24038155 Dialect v1.3

Any later syntax addition, removal or semantic alteration requires a new dialect version.